Day 12: Load Balancing, Serverless, and Event-Driven Architecture

This module covers how to expose your applications to the world and how to connect decoupled services using Google Cloud's serverless ecosystem and messaging middleware.

Section 1: Load Balancing in GKE

In Google Kubernetes Engine (GKE), load balancing is the mechanism that routes traffic from the internet or internal networks to your Pods.

1.1 Internal vs. External Load Balancers

- **External Load Balancer:** Uses a Google Cloud Public IP to route traffic from the internet to your cluster.
- **Internal Load Balancer:** Uses a private IP within your VPC. Ideal for internal microservices that should not be exposed to the public web.

1.2 Ingress and Traffic Flow

- **Service (Type: LoadBalancer):** Provisions a Layer 4 (TCP/UDP) Load Balancer. It is simple but lacks path-based routing.
- **Ingress:** Provisions a Layer 7 (HTTP/S) Load Balancer. It allows for advanced routing, such as sending example.com/api to one service and example.com/web to another.

Section 2: Cloud Run (Serverless Containers)

Cloud Run is a managed compute platform that enables you to run stateless containers that are invocable via web requests or Pub/Sub events.

2.1 Cloud Run vs. GKE

Feature	**Cloud Run**	**GKE**
Management	Fully Managed (Serverless)	Regional/Zonal (Infrastructure)
Scaling	Scale-to-Zero automatically	Scaling nodes and pods (Horizontal Pod Autoscaler)
Cost	Pay-per-request	Pay for provisioned resources
Best For	Web APIs, Microservices, Data Processing	Complex, long-running stateful apps

2.2 Revisions and Traffic Splitting

Every deployment in Cloud Run creates a **Revision**. You can split traffic between revisions (e.g., 90% to V1 and 10% to V2) to perform Canary deployments safely.

Section 3: Cloud Functions & Pub/Sub

This section focuses on asynchronous, event-driven patterns where code execution is triggered by specific occurrences in your cloud environment.

3.1 Cloud Functions

- **HTTP Functions:** Triggered via a standard URL.
- **Event-Driven Functions:** Triggered by background events like a file upload to Cloud Storage or a message arriving in Pub/Sub.

3.2 Pub/Sub Messaging

Pub/Sub (Publisher/Subscriber) is the glue that connects microservices asynchronously.

- **Topic:** The "folder" where messages are sent by publishers.
- **Subscription:** The "queue" where subscribers pull messages from.
- **Fan-out Pattern:** One topic can have multiple subscriptions, allowing one event to trigger multiple independent services simultaneously.

Section 4: Case Studies and Scenarios

Case Study: Event-Driven Image Processing

Scenario: A social media app needs to generate thumbnails whenever a user uploads a high-resolution photo.

Solution:

1. User uploads an image to **Cloud Storage**.
2. Cloud Storage triggers a **Cloud Function**.
3. The Function processes the image and publishes a "Task Complete" message to a **Pub/Sub Topic**.
4. A **Cloud Run** service subscribed to that topic updates the user's profile database.

Section 5: Mini Project - Asynchronous Microservice Flow

This lab demonstrates connecting a Cloud Run service to a Cloud Function using Pub/Sub.

Part A: Setup Pub/Sub

Bash

```
# 1. Create a Topic

gcloud pubsub topics create system-notifications


# 2. Create a Subscription

gcloud pubsub subscriptions create email-service-sub --topic=system-notifications
```

Part B: Deploy Cloud Run (The Publisher)

This service will act as a web gateway that sends a message to Pub/Sub.

Bash

```
# Deploying a sample image that publishes to a topic

gcloud run deploy gateway-service \

   --image=gcr.io/cloudrun/hello \

   --update-env-vars=TOPIC_ID=system-notifications \

   --region=us-central1
```

Part C: Deploy Cloud Function (The Subscriber)

This function triggers automatically whenever a message is published to the topic.

Bash

```
# Deploy a background function triggered by Pub/Sub

gcloud functions deploy process-notification \

   --runtime=python310 \

   --trigger-topic=system-notifications \
```

```
  --entry-point=handler \
  --source=./function-code-dir
```

Part D: GKE Load Balancing (The Ingress)

If you are running the gateway on GKE instead of Cloud Run, use this manifest to expose it externally via an HTTP(S) Load Balancer.

YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: gke-ingress
  annotations:
    kubernetes.io/ingress.class: "gce"
spec:
  rules:
  - http:
      paths:
      - path: /*
        pathType: ImplementationSpecific
        backend:
          service:
            name: gateway-service
            port:
              number: 80
```

Part E: Validation

1. **Traffic Flow:** Send an HTTP request to the Cloud Run/GKE URL.

2. **Message Delivery:** Check the Cloud Function logs (gcloud functions logs read process-notification) to see if the message was received and processed.